

ARQUITETURA E DESENVOLVIMENTO EM JAVA

FASE 1 | AULA 03 -

OPEN/CLOSED PRINCIPLE

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS	5
O QUE VOCÊ VIU NESTA AULA?	9
REFERÊNCIAS	10

EMANIP

O QUE VEM POR AÍ?

Nesta aula, vamos explorar o Princípio do Aberto/Fechado (Open/Closed Principle ou OCP), que é o segundo dos cinco princípios SOLID da programação orientada a objetos e design de software. Vamos entender a importância de projetar nosso código de maneira que ele possa ser estendido sem a necessidade de ser modificado.



HANDS ON

Nesta aula prática, teremos uma introdução ao OCP, definindo e explicando a importância do Princípio do Aberto/Fechado no design de software. Veremos um exemplo de violação do OCP, analisando uma classe que viola esse princípio ao exigir modificações para adicionar novas funcionalidades.

Passaremos então para a refatoração, com um passo a passo de como foi refatorada a classe para aplicar o OCP, utilizando abstrações e interfaces. Por fim, estudaremos os benefícios da aplicação do OCP, com uma discussão sobre as vantagens de seguir o OCP em termos de flexibilidade, manutenção e escalabilidade do código.

SAIBA MAIS

O Princípio do Aberto/Fechado (Open/Closed Principle ou OCP) afirma que o software deve ser aberto para extensão, mas fechado para modificação. Em outras palavras, deve ser possível adicionar novas funcionalidades ao sistema sem alterar o código existente.

A ideia principal do OCP é que um módulo (classe, função, etc.) deve estar aberto para extensão (ou seja, seu comportamento pode ser estendido) sem modificar seu código fonte original. Isso é feito por meio do uso de abstrações, como interfaces ou classes abstratas, que permitem a criação de novas implementações sem alterar o código existente.

Para ficar mais claro, vamos considerar um exemplo de uma aplicação de pagamento que precisa processar dois tipos de pagamentos: cartão de crédito e PayPal.

Violando o OCP

Aqui temos um exemplo de uma classe `PaymentProcessor` que viola o OCP ao incluir lógica para processar diferentes tipos de pagamentos dentro da mesma classe:

```
public class PaymentProcessor {  
    public void processPayment(String paymentType) {  
        if (paymentType.equals("creditCard")) {  
            // Lógica para processar pagamento com cartão de crédito  
            System.out.println("Processing credit card payment...");  
        } else if (paymentType.equals("paypal")) {  
            // Lógica para processar pagamento com PayPal  
            System.out.println("Processing PayPal payment...");  
        }  
    }  
}
```

Código 1 — Classe que viola OCP
Fonte: elaborado pelo autor (2024)

Agora, para aplicar o OCP, vamos refatorar o código utilizando uma interface `Payment` e implementações específicas para cada tipo de pagamento. O

`PaymentProcessor` usará a interface `Payment` para processar pagamentos, permitindo a adição de novos tipos de pagamento sem modificar o código existente.

```
package src;

public interface Payment {

    void process();

}
```

Código 2 — Refatoração
Fonte: elaborado pelo autor (2024)

Agora as nossas outras classes: `CreditCardPayment`, `PayPalPayment` e `PaymentProcessor`.

CreditCardPayment:

```
package src;

public class CreditCardPayment implements Payment {

    @Override
    public void process() {

        // Lógica para processar pagamento com cartão de crédito
        System.out.println("Processing credit card payment...");

    }

}
```

Código 3 — CreditCardPayment
Fonte: elaborado pelo autor (2024)

PayPalPayment:

```
package src;

public class PayPalPayment implements Payment {

    @Override
    public void process() {

        // Lógica para processar pagamento com PayPal
        System.out.println("Processing PayPal payment...");

    }

}
```

Código 4 — PayPalPayment
Fonte: elaborado pelo autor (2024)

PaymentProcessor:

```
package src;  
public class PaymentProcessor {  
    public void processPayment(Payment payment) {  
        payment.process();  
    }  
}
```

Código 5 — PaymentProcessor
Fonte: elaborado pelo autor (2024)

Vamos entender a refatoração. Antes da refatoração, a classe **PaymentProcessor** estava violando o OCP ao acumular lógica para diferentes tipos de pagamento, tornando-se difícil de manter e estender.

Depois da refatoração, nós criamos:

1. Interface **Payment**: define um contrato para processamento de pagamentos, permitindo que novas implementações de pagamento sejam adicionadas sem alterar o código existente.
2. Classe **CreditCardPayment**: implementa a interface **Payment** para processar pagamentos com cartão de crédito.
3. Classe **PayPalPayment**: implementa a interface **Payment** para processar pagamentos com PayPal.
4. Classe **PaymentProcessor**: utiliza a interface **Payment** para processar pagamentos, permitindo a adição de novos tipos de pagamento sem modificar seu código.

Benefícios da refatoração

- **Facilidade de extensão**: novos tipos de pagamento podem ser adicionados facilmente implementando a interface **Payment**, sem modificar o **PaymentProcessor**.

- **Manutenção simplificada:** a lógica de cada tipo de pagamento é isolada em suas próprias classes, facilitando a manutenção e evitando a introdução de bugs em outros tipos de pagamento.
- **Testabilidade:** cada implementação de pagamento pode ser testada independentemente, garantindo que mudanças em uma implementação não afetem outras.

Essa refatoração, portanto, melhora a coesão de cada classe e facilita a manutenção e evolução do código ao longo do tempo, seguindo o Princípio do Aberto/Fechado.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, exploramos o Princípio do Aberto/Fechado (Open/Closed Principle ou OCP), o segundo dos cinco princípios SOLID da programação orientada a objetos e design de software.



REFERÊNCIAS

FREECODECAMP. **Open-Closed Principle – SOLID Architecture Concept Explained**. 2023. Disponível em: <https://www.freecodecamp.org/news/open-closed-principle-solid-architecture-concept-explained/>. Acesso em: 06 ago. 2024.

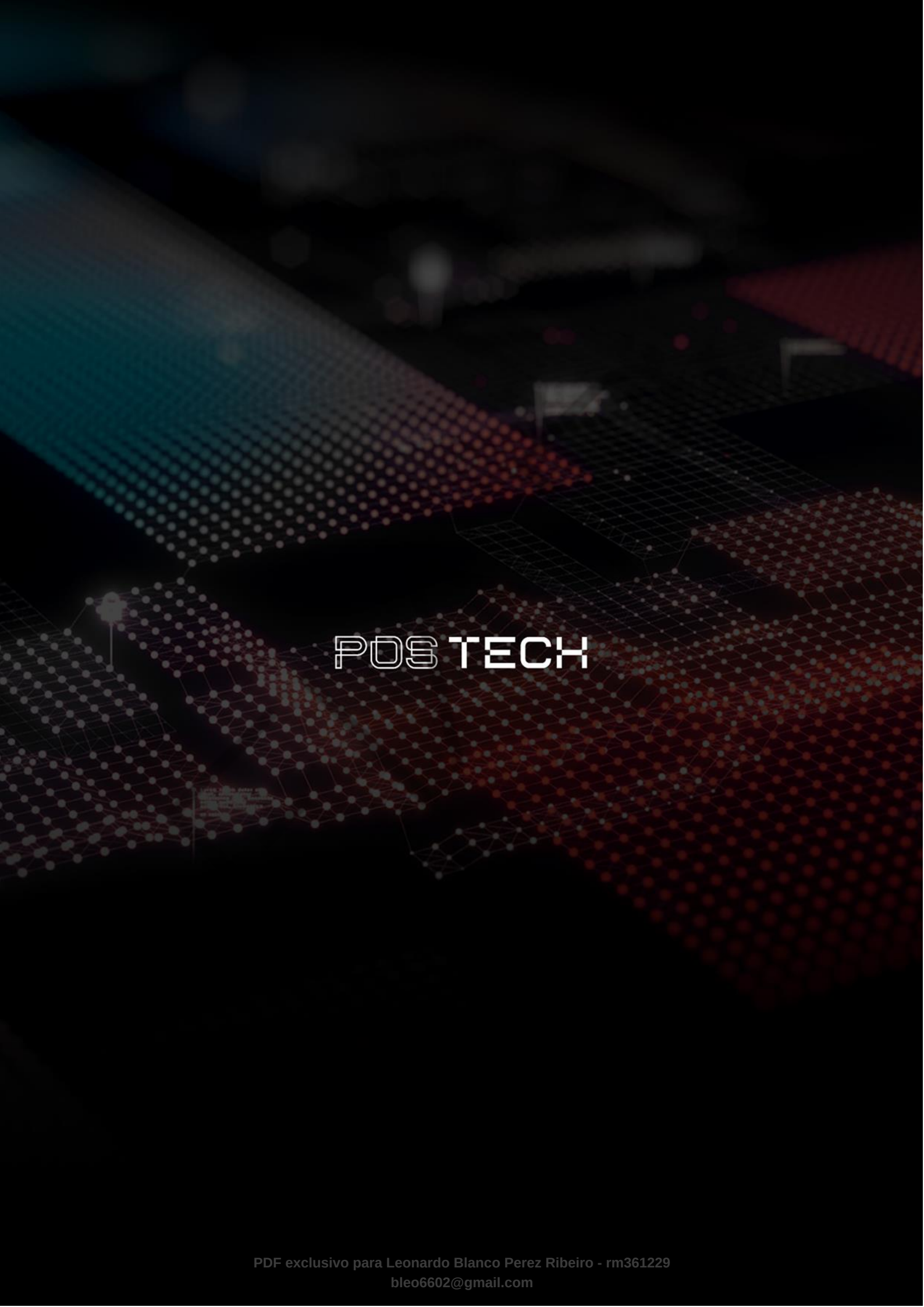
MEDIUM. **SOLID — OCP — Open Closed Principle**. 2019. Disponível em: <https://medium.com/@tbaragao/solid-ocp-open-closed-principle-600be0382244>. Acesso em: 06 ago. 2024.

PIRES, E. **SOLID – Open Closed Principle – OCP**. 2013. Disponível em: <https://www.eduardopires.net.br/2013/05/open-closed-principle-ocp/>. Acesso em: 06 ago. 2024.

PALAVRAS-CHAVE

Palavras-chave: SOLID; OPEN CLOSED PRINCIPLE; OCP.

EMEND



POSTECH